

Bitcoin Multicast TXID Shard Bit Extraction

Describing a method to load balance massive flows of transactions by splitting work.

ELI5 — Simple Explanation

Imagine a post office with 4096 sorting bins.

Every Bitcoin transaction has an ID — a long random-looking number (the TXID). We want to quickly decide which of the 4096 bins it belongs in, without doing any expensive math.

Here's the trick: look at just the *first 4 bytes* of the TXID, line them up in order (that's the big-endian part), then grab only the *first 12 bits* of those bytes (that's the right-shift). Those 12 bits give a number from 0 to 4095 -- the bin number.

TXID bytes	[A3] [F7] [C2] [E1] ...
Read as uint32	0xA3F7C2E1
Top 12 bits	0xA3F = 2623 ← shard #2623 of 4096

Because TXIDs are random hashes, every bin gets roughly the same number of transactions.
Fast, fair, and no division required.

Source

```
shardBits = 12

groupIndex = binary.BigEndian.Uint32(txid[0:4]) >> (32 - shardBits)
```

This extracts the top 12 bits of the big-endian uint32 formed by the first 4 bytes of the TXID, producing a shard index in the range 0–4095.

Step-by-Step Walkthrough

Sample TXID: a3f7c2e1...

Step 1 — Take first 4 bytes (highlighted)



Step 2 — Assemble as big-endian uint32: **0xA3F7C2E1 = 2,750,923,489**

Step 3 — View all 32 bits; top 12 are the shard index



Step 4 — Right-shift by 20 (= 32 - 12), keeping top 12 bits:

groupIndex

262

0xA

shard 2623 of 4096

3

3F

Big-Endian vs Little-Endian Comparison

The endianness of the Uint32 read changes *which* bytes of the TXID contribute to the shard index. Big-endian places `txid[0]` as the most significant byte — so it drives the shard bits. Little-endian would put `txid[3]` there instead.

TXID prefix	Encoding	uint32 (hex)	groupIndex	shard of 4096
a3f7c2e1...	BigEndian	0xA3F7C2E1	2623	2623/4095
	LittleEndian	0xE1C2F7A3	3612	3612/4095
000000001a2b...	BigEndian	0x00000000	0	0/4095
	LittleEndian	0x00000000	0	0/4095
ffffffff0000...	BigEndian	0xFFFFFFFF	4095	4095/4095
	LittleEndian	0xFFFFFFFF	4095	4095/4095
deadbeef1234...	BigEndian	0xDEADBEEF	3562	3562/4095
	LittleEndian	0xEFBEADDE	3835	3835/4095

Bit-Level View — All Samples

Each row shows all 32 bits of the big-endian uint32. Blue = shard index bits (top 12). Gray = discarded (bottom 20).



a3f7c2e1... → groupIndex = 2623 (0xA3F)



000000001a2b... → groupIndex = 0 (0x000)



fffffff0000... → groupIndex = 4095 (0xFFF)



deadbeef1234... → groupIndex = 3562 (0xDEA)

Why This Is a Good Sharding Function

Uniform distribution. TXIDs are double-SHA256 hashes, so their bytes are effectively uniform random. The top 12 bits will land uniformly across all 4096 shards — no hot spots.

Zero modulo cost. A right-shift is a single CPU instruction. Compared to a modulo operation (integer division), this is ~5–10x faster at high throughput.

Deterministic and stateless. Any node seeing the same TXID independently computes the same shard index — no coordination required. Critical for multicast fan-out.

Endianness is load-bearing. BigEndian ensures txid[0] (the first wire byte) drives the shard bits. A LittleEndian read would use txid[3] instead, producing a completely different — and potentially skewed — distribution if TXIDs had any byte-positional bias.

Tunable range. Changing shardBits changes the shard count: 10 bits → 1024 shards, 12 bits → 4096 shards, 14 bits → 16384 shards. Only the shift constant changes.

Shard count by shardBits value

shardBits	Shards (2^n)	Index range	Right-shift
8	256	0 – 255	>> 24
10	1,024	0 – 1,023	>> 22

12	4,096	0 - 4,095	>> 20
14	16,384	0 - 16,383	>> 18
16	65,536	0 - 65,535	>> 16

Row highlighted in blue = current config (shardBits = 12)